Analysis of Massive Data Sets

http://www.fer.hr/predmet/avsp

Izv. prof. dr. sc. Marin Šilić

Izv. prof. dr. sc. Goran Delač

Izv. prof. dr. sc. Klemo Vladimir

Doc. dr. sc. Adrian Satja Kurdija

Faculty of Electrical Engineering and Computing
Consumer Computing Laboratory

Copyright © 2026 CCL : Analysis of Massive Data Sets

Detection of near-duplicate (textual) documents using the *simhash* algorithm

Copyright © 2026 CCL : Analysis of Massive Data Sets

Outline

- Motivation
- Fingerprinting
  - Simhash algorithm
  - Rabin's rolling hash function
- IR/NLP basics
- Scalable queries
- Similar methods
  - Random Indexing
  - Image hashing
- Literature

Copyright © 2026 CCL : Analysis of Massive Data Sets

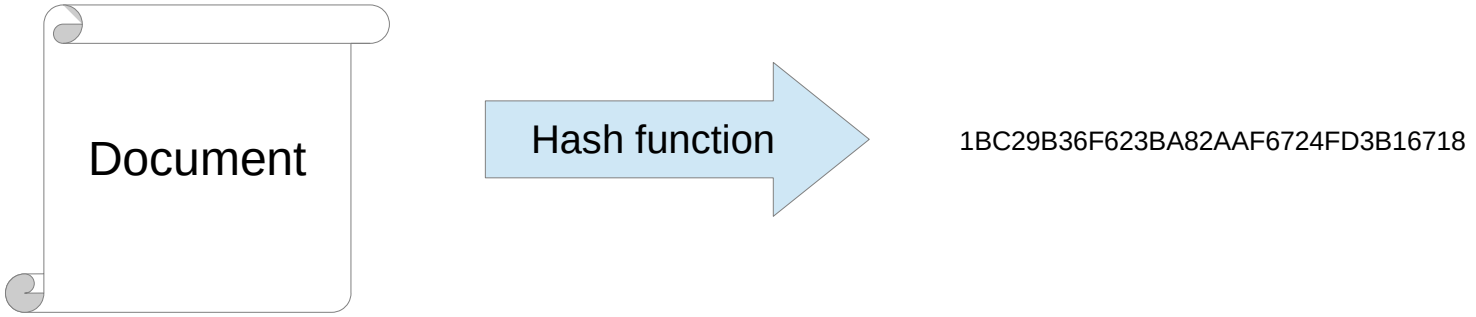
Motivation

- Near-duplicate documents
  - Versioning
    - Different versions of single document
      - Revisions, different file formats, ...
  - Mirroring
    - published in more than one location
  - Plagiarism
    - Exact or “processed” copy
  - Malware
    - Viruses, spam, …
- Scalability
  - Documents and document respositories are large

Copyright © 2026 CCL : Analysis of Massive Data Sets

Exact copy analysis

- Checksumming
  - Cryptographic hash functions
  - MD5, SHA1, SHA2, ...



  - Catches the smallest edit
    - Great for detection of **exact** copies
    - Not so good for near-duplicate detection
      - Even the smallest change will result in totally different digest

Copyright © 2026 CCL : Analysis of Massive Data Sets

Near-duplicate detection

- Two families of methods
  - 1. Ranking
    - Information retrieval techniques [*1]
    - High-dimensional vectors manipulation
  - 2. *Fingerprinting*
    - Document hashing [*2]
    - Dimensionality reduction

Copyright © 2026 CCL : Analysis of Massive Data Sets

Fingerprinting

- Similarity preserving hashing
  - X, set of inputs
  - d_x, distance function on X
    - x_1, x_2 elements of X
  - similarity preserving hash function
    - **h: X → Y**
    - |Y| < |X|
  - d_y, distance function on Y

Copyright © 2026 CCL : Analysis of Massive Data Sets

Fingerprinting

- Similarity preserving hashing
  - Similar inputs have similar hashes

$$\text{if } d_x(x_1, x_2) < \varepsilon_x, \text{ then}$$
$$d_y(h(x_1), h(x_2)) < \varepsilon_y$$

- Illustrative example
  - `h("1234") = 0xaaaf  h("5678") = 0xb115`
  - `h("1235") = 0xaaae`

Copyright © 2026 CCL : Analysis of Massive Data Sets

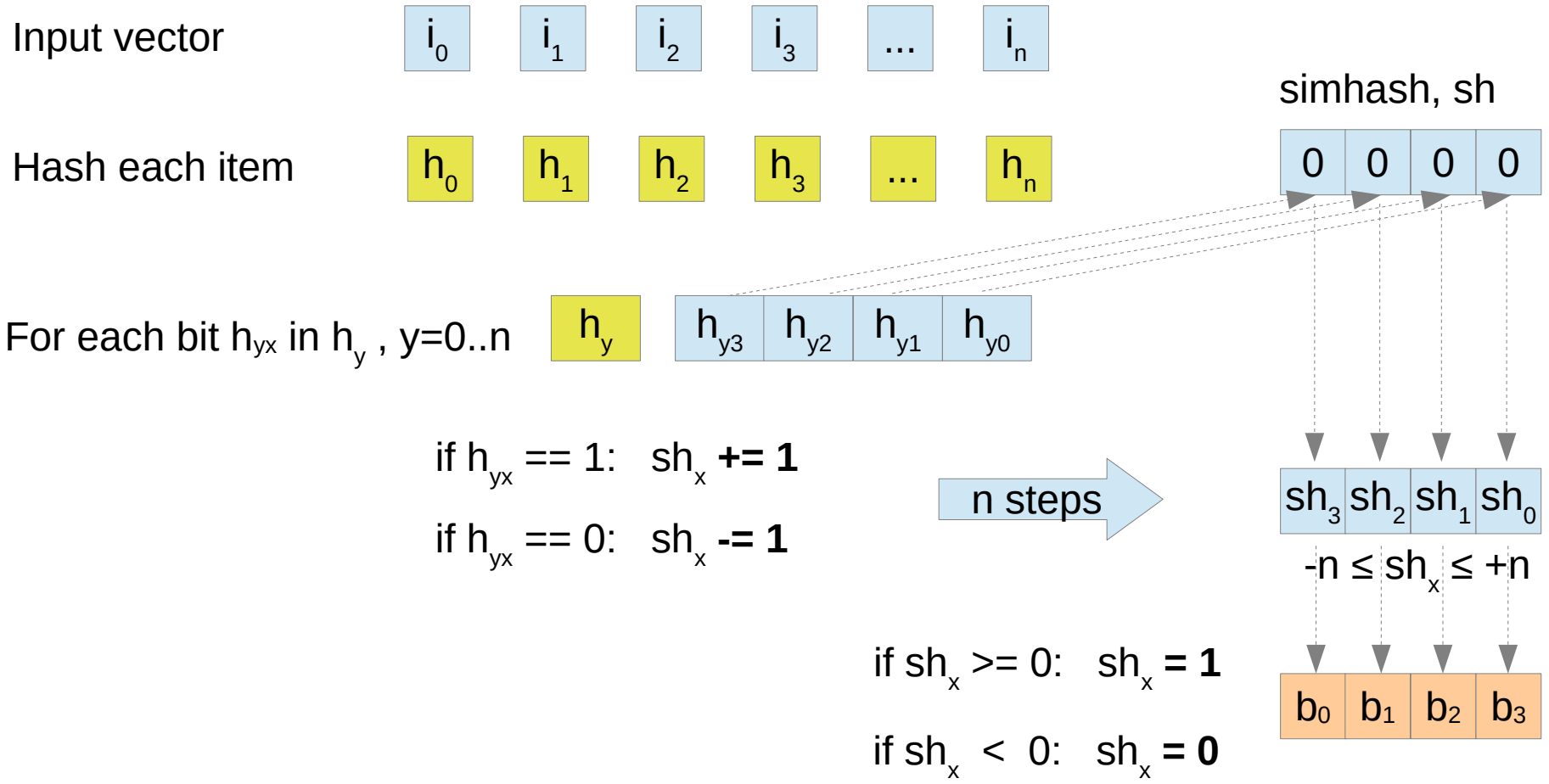
Fingerprinting

- **Simhash** algorithm [*3]
  - Author: M. Charikar, 2002
  - Fingerprinting technique
    - Fingerprints of near-duplicates differ in a small number of bit positions (**hamming** distance)
  - Dimensionality reduction
    - Maps high-dimensional vectors to small-sized fingerprints (f-bits)
  - Input
    - High-dimensional vector (strings, numbers, …)
  - Output
    - f-bit fingerprint
  - Fingerprint size (f-bits)
    - f is small and arbitrary
    - eg. f=64 for web sites

Copyright © 2026 CCL : Analysis of Massive Data Sets

Simhash

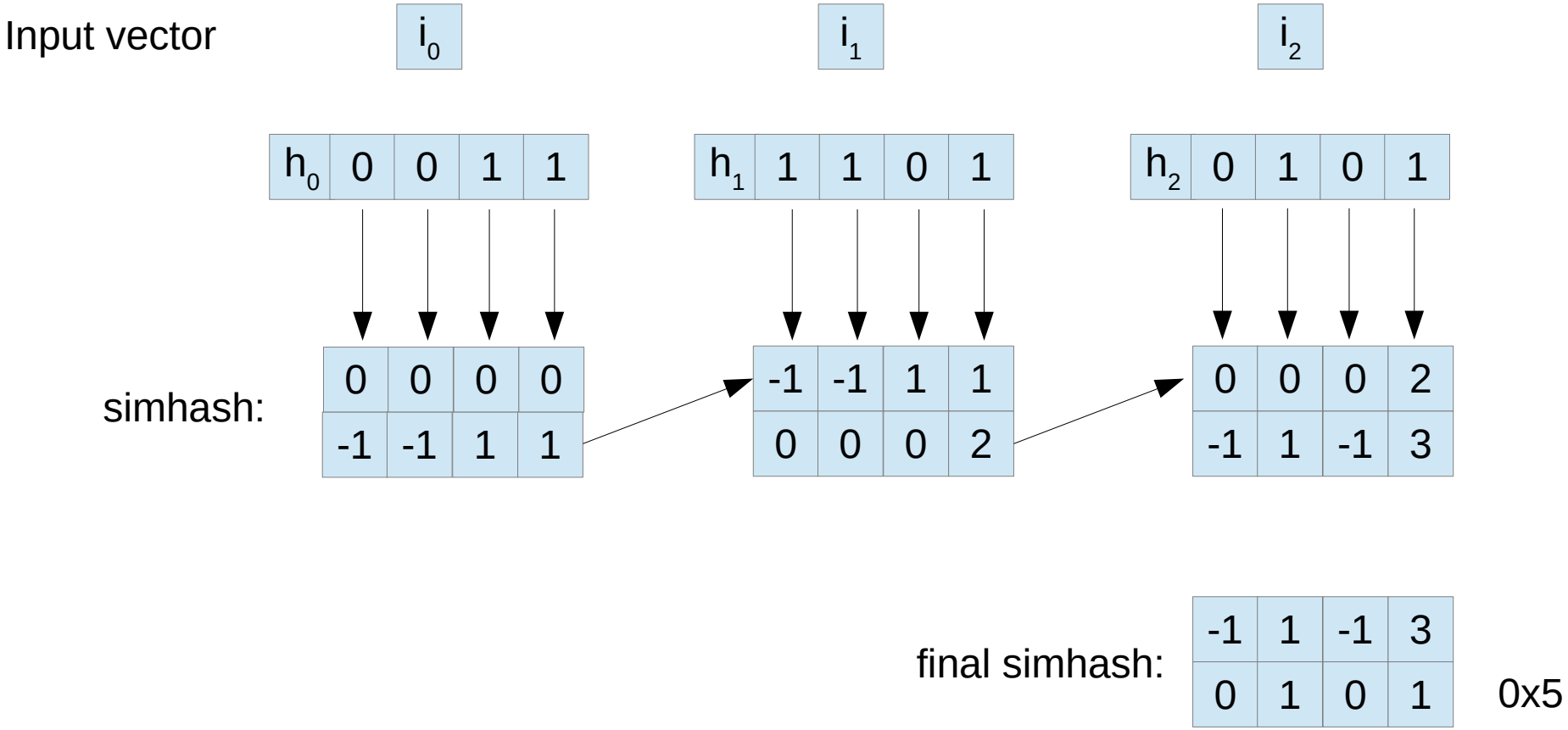
- Simhash computation, f=4



Copyright © 2026 CCL : Analysis of Massive Data Sets

Simhash

- Simhash example, f=4

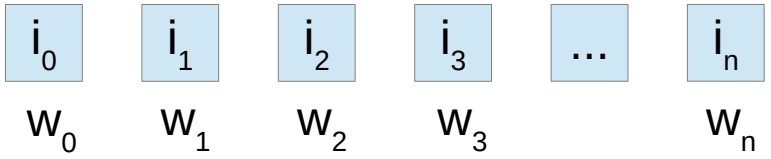


Copyright © 2026 CCL : Analysis of Massive Data Sets

Simhash

- Weighted Simhash computation
  - Assign weight factor to each feature

Input vector



if h_{yx} == 1: sh_x **+= w_y**

if h_{yx} == 0: sh_x **-= w_y**

Copyright © 2026 CCL : Analysis of Massive Data Sets

Simhash

- Choice of hash function *h*
  - Uniform distribution
  - Fast
  - Candidates
    - Cryptographic hash functions
      - MD5 (128-bit), SHA-1 (256-bit)
      - Problem: cryptographic hashing is slow
    - Rolling hash functions
      - Input is hashed by moving window element by element

Copyright © 2026 CCL : Analysis of Massive Data Sets

Simhash

- Choice of hash function *h*
  - Rabin rolling hash
    - Used in Rabin-Karp string searching algorithm
    - Input: string
    - Output: hash - number
  - Computation
    - substring coded as a number with base **d**
      - d = total number of possible characters
    - Coded string at position *i*:

$$x_i = s[i]d^{k-1} + s[i+1]d^{k-2} + \ldots + s[i+k-1]$$

Copyright © 2026 CCL : Analysis of Massive Data Sets

Simhash

- Choice of hash function *h*
  - Rabin rolling hash
    - Example: k=4, d=32
    - i=0

0	1	2	3	4		5	6	7	8	9
l	o	r	e	m		i	p	s	u	m
l	o	r	e							

h(“lore”) = $h(x_o) = x_o \bmod q$ *[q is some large prime number]*

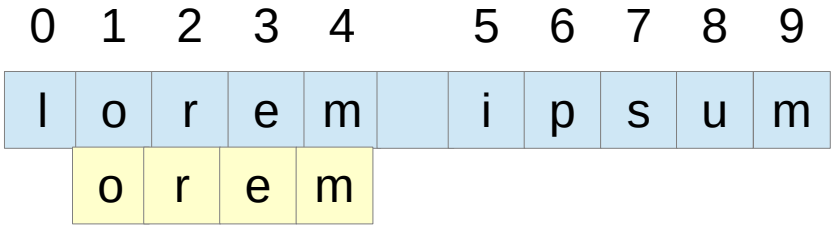
$x_o = \text{int}(\text{‘l’})32^3 + \text{int}(\text{‘o’})32^2 + \text{int}(\text{‘r’})32 + \text{int}(\text{‘e’})$

x_{i+1}=?

Copyright © 2026 CCL : Analysis of Massive Data Sets

Simhash

- Choice of hash function *h*
  - Rabin rolling hash
    - Example: k=4, d=32
    - i=1



h(“orem”) = $h(x_1) = x_1 \bmod q$

$x_1 = (x_0 - \text{int}(\text{‘l’})d^3)d + \text{int}(\text{‘m’})$

$\mathbf{x_{i+1} = (x_i - s[i]d^{k-1})d + s[i+k]}$

Copyright © 2026 CCL : Analysis of Massive Data Sets

Simhash

- Choice of hash function *h*
  - Rabin rolling hash
    - Fingerprints represented using polynomials
    - Computes hash value of the next string from the previous one
      - Constant number of operations
      - Independent of string length

Copyright © 2026 CCL : Analysis of Massive Data Sets

IR – A brief excursion

- *How to represent documents? How to calculate simhash weights?*
- **IR – Information retrieval**
  - Models and algorithms for retrieving information from document repositories
- **Statistical NLP – Natural Language Processing**
  - Vector space model
    - Each document is represented as a vector in a high-dimensional space
    - Dimensions are “terms”, values are term frequencies
    - Semantic proximity ↔ Spatial proximity
  - N-gram model for terms definition
    - 1-gram, 2-gram, 3-gram, ...

Copyright © 2026 CCL : Analysis of Massive Data Sets

IR – A brief excursion

- N-gram model
  - Tokenization examples

“lorem ipsum dolor sit amet”

1-gram word

“lorem”, “ipsum”, “dolor”, “sit”, “amet”

2-gram (bigram) words

“lorem ipsum”, “ipsum dolor”, “dolor sit”, “sit amet”

3-gram (trigram) characters

“lor”, “ore”, “rem”, “em ”, “m i”, …

Copyright © 2026 CCL : Analysis of Massive Data Sets

IR – A brief excursion

- N-gram model
  - Document collection is usually represented in a matrix form
    - Rows – documents
    - Columns – terms (1-word)

```
D1 = “lorem ipsum dolor sit lorem amet sit”

D2 = “lorem ipsum adipiscing ipsum elit amet ipsum”
```

	adipiscing	amet	dolor	elit	ipsum	lorem	sit
D1	0	1	1	0	1	2	2
D2	1	1	0	1	3	1	0

Copyright © 2026 CCL : Analysis of Massive Data Sets

IR – A brief excursion

- Calculating similarity between documents
  - Document similarity = vector similarity!
  - Usually, ***cosine*** similarity is calculated
- Adjusting term weights (**tf-idf** model)
  - $TF_{i,j}$ = term frequency of w_i in document D_j
  - Some *tf* schemes:
    - $tf(TF_{i,}) = TF_i$
    - $tf(TF_{i,}) = log(1 + TF_{i,})$
  - Problem - terms that are rare vs. terms that are common
    - N – total number of documents
    - DF_i = number of documents that contain term w_i
    - $idf_i(N, DF_i) = log(1 + N/DF_i)$
  - Final tf-idf scheme
    - $tf\text{-}idf(TF_{i,,}\ N, DF_i) = tf(TF_{i,}) * idf(N, DF_i)$

Copyright © 2026 CCL : Analysis of Massive Data Sets

IR – A brief excursion

- N-gram model
  - Number of dimensions = number of terms
    - Small N: dissimilar documents appear similar
    - Large N: similar documents appear dissimilar
  - Example:
    - Vocabulary size = 20000
    - 2-gram dimensionality ~ 20k x 20k = 400 mil.
    - 3-gram dimensionality ~ $8 * 10^{12}$
  - Reducing dimensionality
    - Removing “stopwords” - words that are common in every document (e.g. “the”, “it”, “an”, …)
    - Stemming – reducing words to a stem (base or root form)
      - Problems: two different words can have equal stem and stems can be unintelligable to users

Copyright © 2026 CCL : Analysis of Massive Data Sets

Fast Queries

- **F**, collection of f-bit fingerprints
- **Q**, query
  - single or set of fingerprints

- Task
  - identify whether **Q** differs from any of the fingerprints in ***F*** in at most **k** bits

Copyright © 2026 CCL : Analysis of Massive Data Sets

Fast Queries

- Example numbers
  - 8B 64-bit fingerprints = 64GB
  - Online query
    - Q = single fingerprint
    - Restriction: few milliseconds
  - Batch query
    - Q = set of fingerprints
      - e.g. |Q| = 1M
    - Restriction: ~100seconds
      - 1B queries per day

Copyright © 2026 CCL : Analysis of Massive Data Sets

Fast Queries

- 1. approach
  - Build sorted table of **F**
  - Build list **Q'** with all fingerprints whose Hamming distance from **Q** is at most **k**

Q
100

k=1

Q'
1 0 0
1 0 1
1 1 0
0 0 0

F
0 0 0
0 0 1
0 1 0
0 1 1
1 0 0
1 0 1
1 1 0
1 1 1

  - 64-bit fingerprints (k=3)
    - $|Q'| = \binom{64}{3} = 41664$

Copyright © 2026 CCL : Analysis of Massive Data Sets

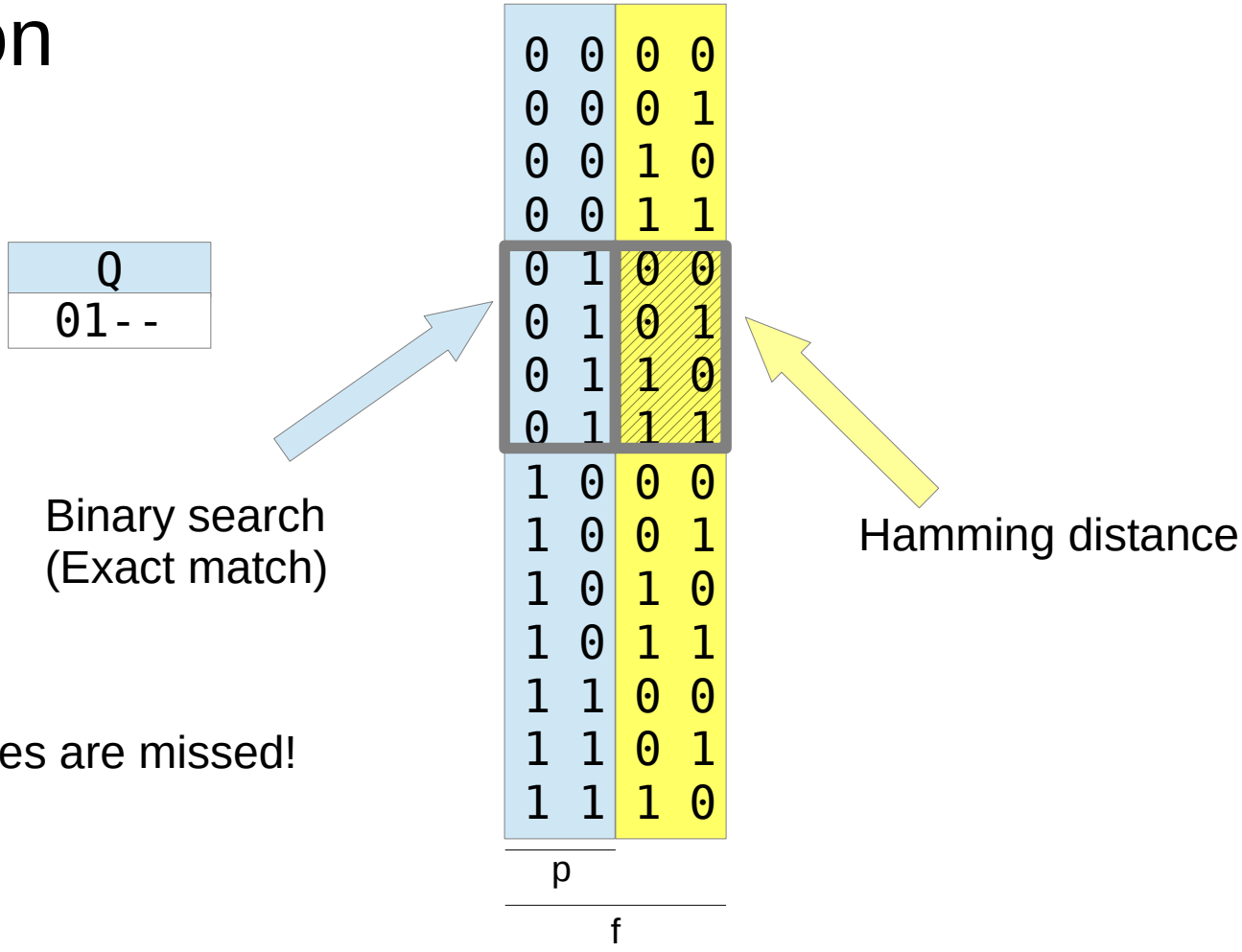
Fast Queries

- 2. approach
  - 1. Build sorted table of **F**
  - 2. Find set of fingerprints (**F'**) that have equal most significant part (p bits)
    - Sorted table – binary search O(p)
  - 3. Check Hamming distance for each fingerprint in **F'**
  - This approach will locate all fingerprints in **F** that differ in at most k bits
    - **BUT, restricted to (only) least significant f-p bits!**

Copyright © 2026 CCL : Analysis of Massive Data Sets

Fast Queries

- Illustration

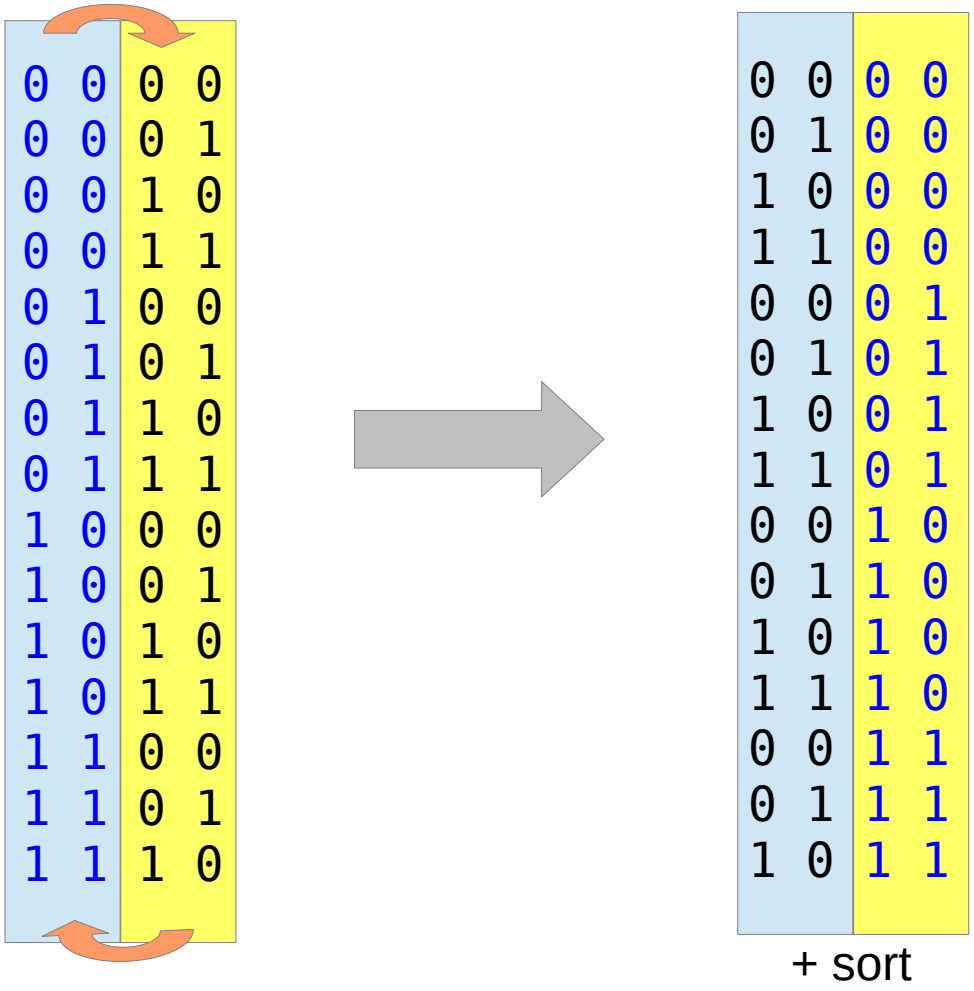


Since p < f:
some true positives are missed!

Copyright © 2026 CCL : Analysis of Massive Data Sets

Fast Queries

- Increasing precision/recall
  - Generate additional table
    - Reversed positions



Copyright © 2026 CCL : Analysis of Massive Data Sets

Fast Queries

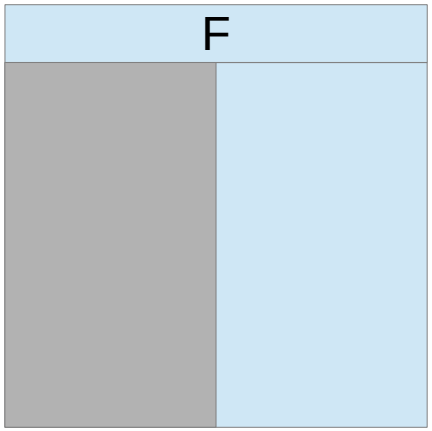
- Solution
  - Build additional tables
    - Each with *different permutation* of bits
    - Every table has different set of significant bits

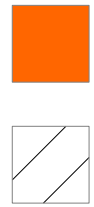
- Algorithm for fast (online) queries
  - Build **t** sorted tables of fingerprints: T_1, T_2, …, T_t
  - Each table T_i also contains
    - p_i – number of significant bits
    - Π_i – random permutation
  - *Every fingerprint in T_i is permuted with permutation Π_i*

Copyright © 2026 CCL : Analysis of Massive Data Sets

Fast Queries

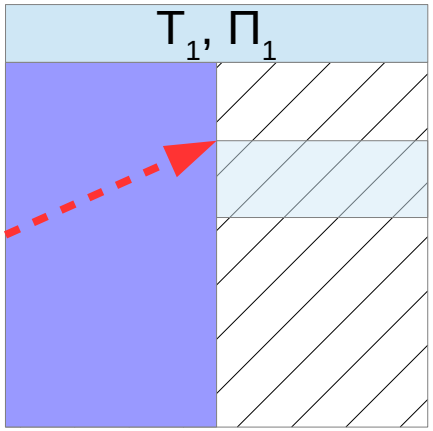
- Illustration
  - Build tables
  - Query in parallel

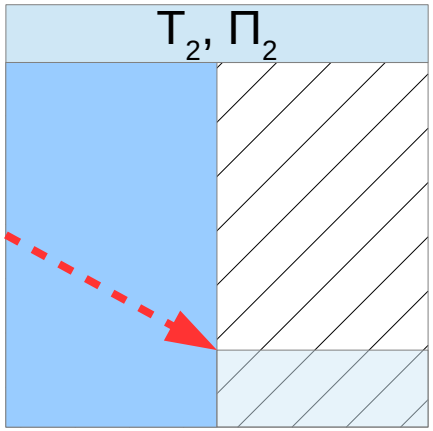


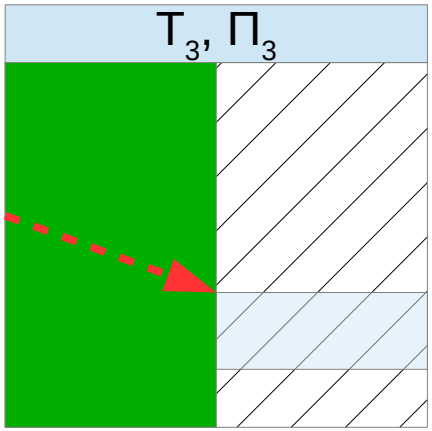


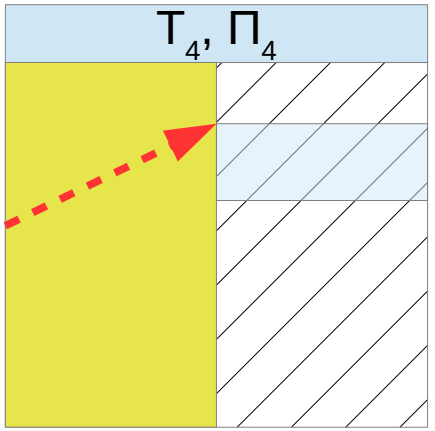
Binary search

Hamming









Copyright © 2026 CCL : Analysis of Massive Data Sets

Fast Queries

- For given **Q** and **k**
  - Read each table (in parallel)
    - 1. Get fingerprints in T_i whose significant p_i bits match the significant p_i bits of $\Pi_i(Q)$
      - T'_i
      - $O(p_i)$ steps
    - 2. For each fingerprint in T'_i, check if it's Hamming distance is at most k bits from $\Pi_i(Q)$

Copyright © 2026 CCL : Analysis of Massive Data Sets

Fast Queries

- Example with t=20, f=64, k=3, |F| = 8B (2^{34})
  - Split f into 6 blocks (4x11 + 2x10 bits)
  - Select 3 out of 6 blocks $\binom{6}{3}$ = 20 ways
  - Arrange those blocks as significant bits
  - p = sum of those bits
    - 31, 32, or 33
  - On average query returns 2^{34-31}=8 fingerprints

Copyright © 2026 CCL : Analysis of Massive Data Sets

Fast Queries

- t and p parameters
  - t ~ p
  - Query time ~ 1/p
  - Storage requirements ~ p
  - Space/time tradeoff
    - Analytical solution for t [*2]

Copyright © 2026 CCL : Analysis of Massive Data Sets

Experimental results

- Detecting near-duplicate web pages
  - Web-crawling in Google
- Database
  - 8B fingerprints, k=1..10
- Manually tag experimental data set
  - True/false positive/negative
- Precision/recall graph
  - Precision: #tp / # returned results
  - Recall: #tp / # expected results

Copyright © 2026 CCL : Analysis of Massive Data Sets

Experimental results

- Precision
  - #tp / # returned results
  - Percentage of relevant items in the returned set
  - Example
    - Qr = [**d3**, **d1**, d4, **d2**, d5], $relevant_{Qr}$ = {d1, d2, d3}
    - P = 3/5=60%
- Recall
  - #tp / # expected results
  - Percentage of all relevant documents in the collection that is in the returned set
  - Example:
    - Qr = [**d3**, **d1**, d4, **d2**, d5], $relevant_{collection}$ = {d1, d2, d3, d7, d8, d9, d10}
    - R = 3/7=43%

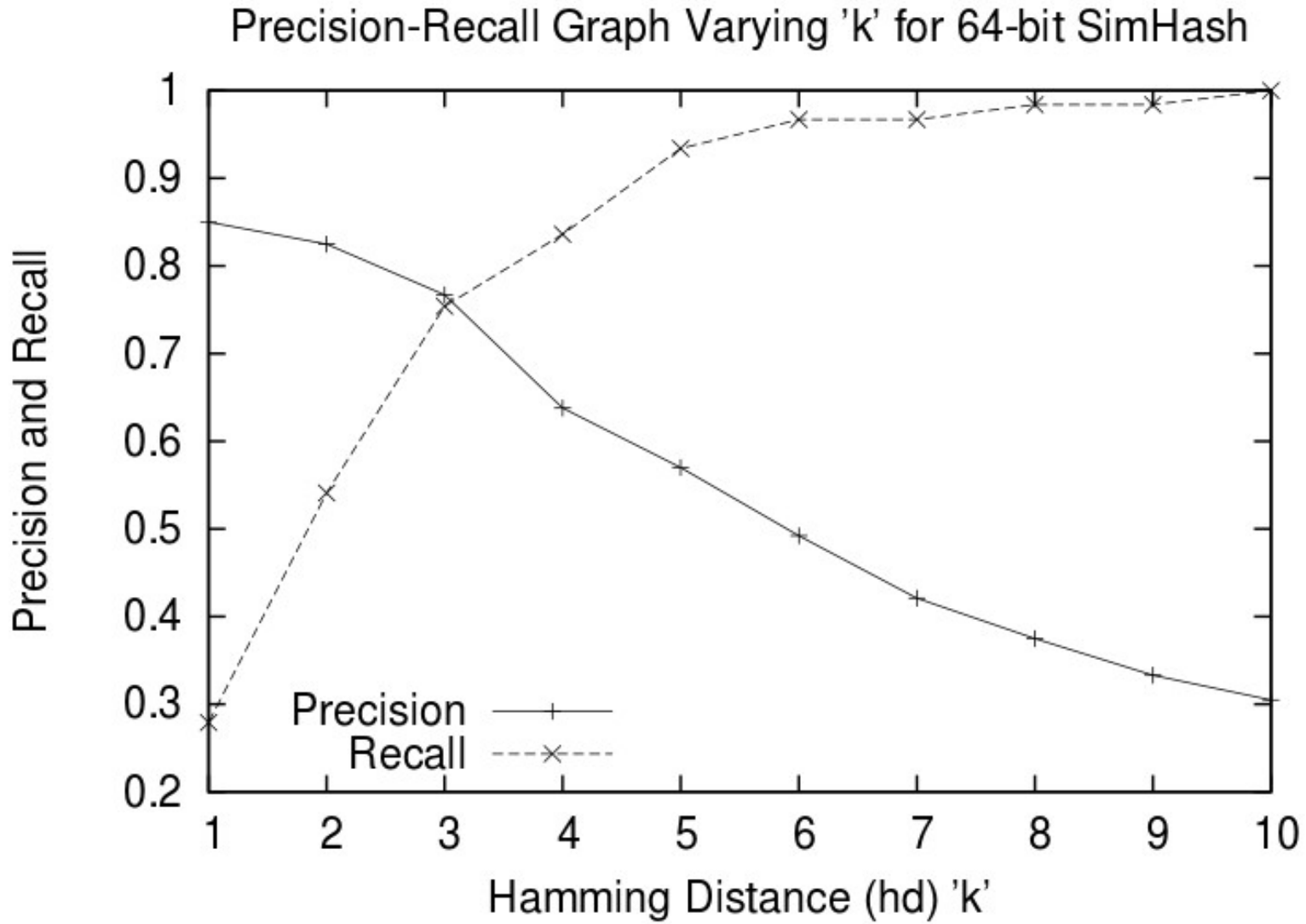
Copyright © 2026 CCL : Analysis of Massive Data Sets

Experimental results

- Two extreme cases:
  - Qr is whole collection
    - Recall is 100%
    - Precision is low
  - Qr is small
    - Precision is high
    - Recall is low
- F-score
  - Combination of Precision and Recall
  - $F_1 = 2PR / (P + R)$ (harmonic mean)

Copyright © 2026 CCL : Analysis of Massive Data Sets

Experimental results



Copyright © 2026 CCL : Analysis of Massive Data Sets

Similar methods

- Random Projection / Random Indexing [*5]
  - Dimentionality reduction technique
  - Input
    - d-dimenional data
  - Output
    - k-dimensional data
    - k << d
  - Procedure
    - Construct a random d *x k* matrix *R*
    - Project d-dim input data to a smaller k-dim space by multiplying input data with matrix *R*

Copyright © 2026 CCL : Analysis of Massive Data Sets

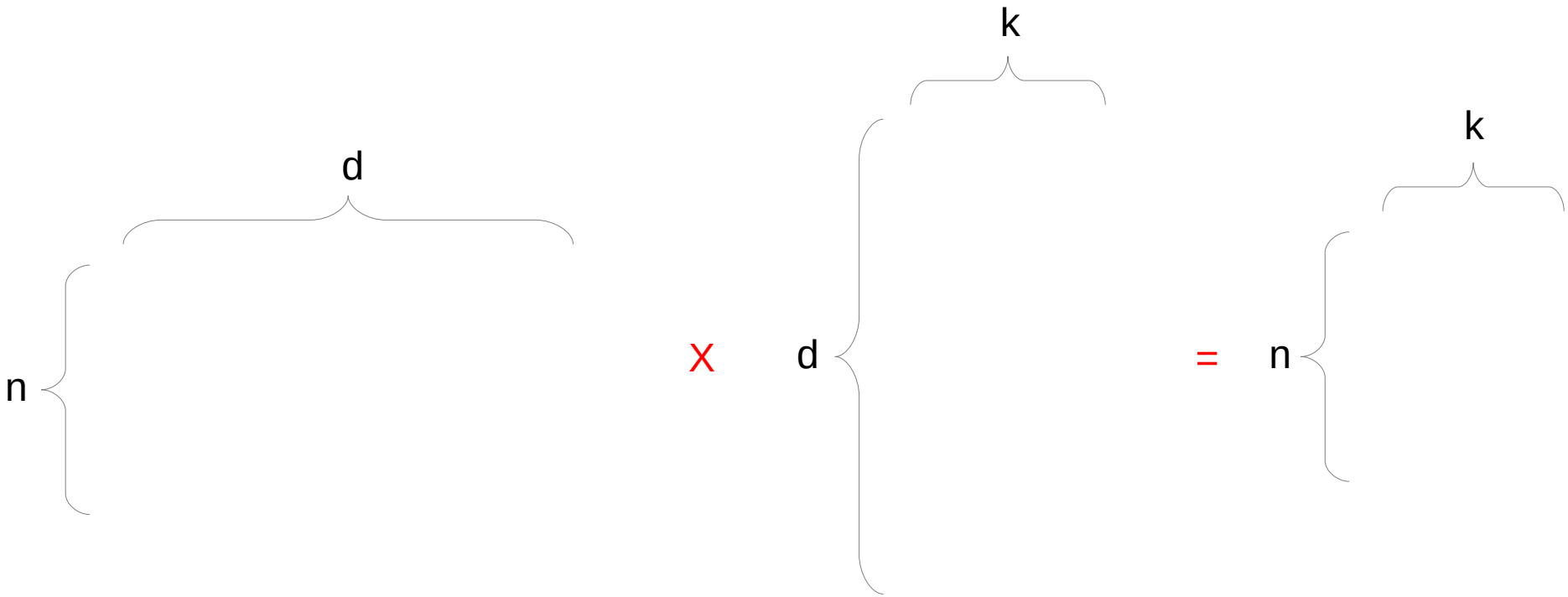
Similar methods

- Random Projection / Random Indexing
  - Random d *x k* matrix *R*
    - *Johnson-Lindenstrauss Lemma*
      - If points in a vector space are projected onto a randomly selected subspace, then the distances between the points are approximately preserved
    - Ideally, *R* should be orthogonal
      - For high-dimensional data, random directional vectors are *good enough*
    - *R* is usually constructed as sparse Gaussian distributed random matrix

Copyright © 2026 CCL : Analysis of Massive Data Sets

Similar methods

- Random Projection / Random Indexing



n = # of input documents
d = # of features
k = # of dim. reduced features

Copyright © 2026 CCL : Analysis of Massive Data Sets

Similar methods...

- PCA (Principal Component Analysis)
- LDA (Linear Discriminant Analysis)
- MF (Matrix Factorization)
  - Stohastic Gradient Descent (SGD)
- Autoencoder
  - usually as neural models
- Data sketching
  - Data streams lecture

Copyright © 2026 CCL : Analysis of Massive Data Sets

Similar methods

- Near-duplicate image detection
  - Simple fingerprint construction from global image descriptiors
    - e.g. A-hash (average hash)*
      - Reduce image size to 8x8
      - Convert to greyscale
      - Compute average of all colors
      - Construct 64-bit hash: 1 if color is above the average, else 0
    - Good enough for very similar images
  - Bag-of-Visual-Words *5
    - Sensitive to more aggressive tranformations
    - Detect interesting regions (e.g. DoG – Difference of Gaussians)
    - Extract features from regions (e.g. SIFT – Scale Infariant Feature Transform)
    - Build a visual vocabulary by vector quantization of feature descriptors
    - Reduce the dimensionality (Minhash – next week!)

* https://pypi.python.org/pypi/ImageHash

Copyright © 2026 CCL : Analysis of Massive Data Sets

Literature

- [1] Hoad, Timothy C., and Justin Zobel. **"*Methods for identifying versioned and plagiarized documents.*"** Journal of the American society for information science and technology 54, no. 3 (2003): 203-215.
- [2] Manku et al. **"*Detecting near-duplicates for web crawling.*"** In Proceedings of the 16th international conference on World Wide Web, pp. 141-150. ACM, 2007. (**REQUIRED READING!**)
- [3] Charikar, Moses S. **"Similarity estimation techniques from rounding algorithms."** In Proceedings of the thiry-fourth annual ACM symposium on Theory of computing, pp. 380-388. ACM, 2002.
- [4] Henzinger, Monika. "**Finding near-duplicate web pages: a large-scale evaluation of algorithms**." Proceedings of the 29th annual international ACM SIGIR conference on Research and development in information retrieval. ACM, 2006.
- [5] Bingham, Ella, and Heikki Mannila. **"Random projection in dimensionality reduction: applications to image and text data."** Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining. ACM, 2001.
- [6] Sivic, Josef, and Andrew Zisserman. **"Video google: A text retrieval approach to object matching in videos."** iccv. Vol. 2. No. 1470. 2003.

Copyright © 2026 CCL : Analysis of Massive Data Sets